

CO₂ Solubility and Density Calculations User Manual

June 2026

Instructions

1

Open the notebook file (.ipynb) in preferred Python environment

2

Review the User Inputs and Experimental Data sections

3

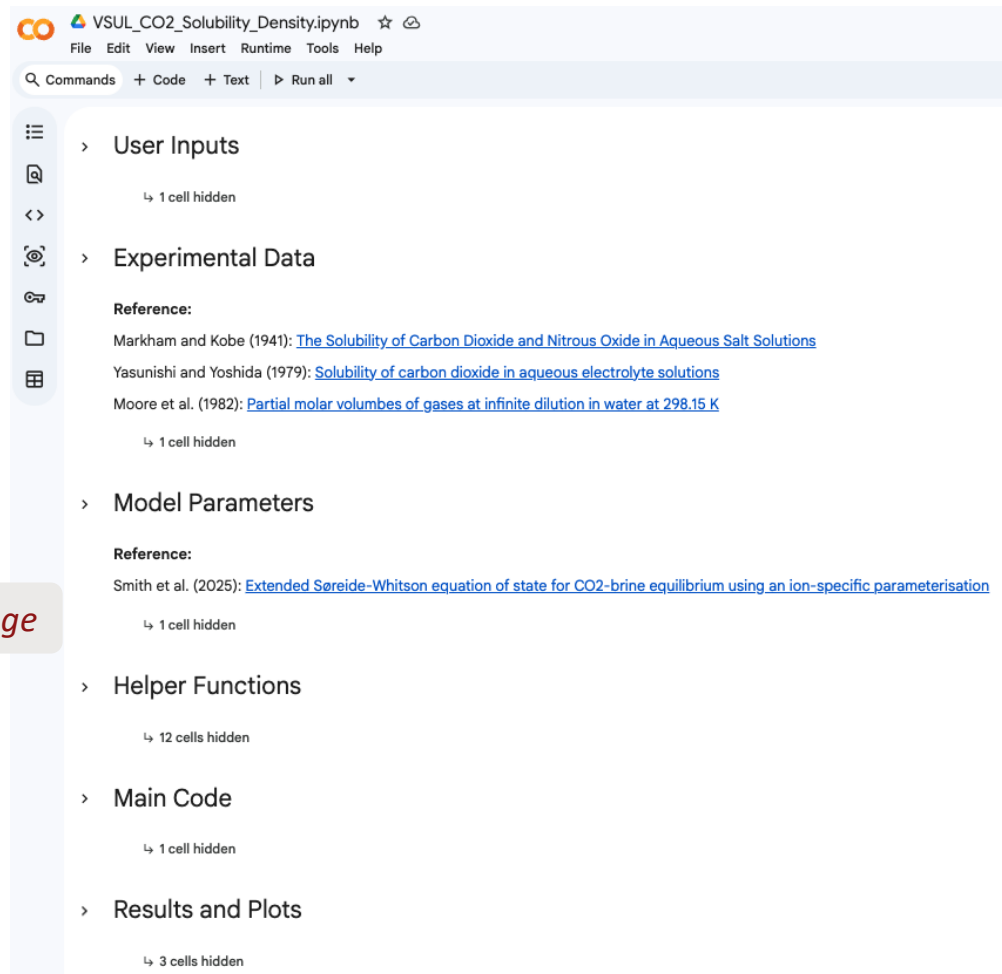
Press 'Run all' and review the generated results and plots

Notebook Overview

The Python notebook contains:

- User Inputs
- Experimental Data
- Model Parameters
- Helper Functions
- Main Code
- Results and Plots

No need to change



User Inputs

```
# ----- Temperature and Pressure -----
```

```
T = 25 + 273.15 # K
```

```
P = 0.101325 * 1e6 # Pa
```

```
# ----- Ion concentrations in the brine in mol/kg -----
```

```
# [Na, K, Mg, Ca, Cl, SO4, NO3]
```

```
ion_conc_arr = [
```

```
    [0.0, 0, 0, 0, 0.0, 0, 0],
```

```
    [0.5, 0, 0, 0, 0.5, 0, 0],
```

```
    [1.0, 0, 0, 0, 1.0, 0, 0],
```

```
    [1.5, 0, 0, 0, 1.5, 0, 0],
```

```
    [2.0, 0, 0, 0, 2.0, 0, 0],
```

```
    [2.5, 0, 0, 0, 2.5, 0, 0],
```

```
    [3.0, 0, 0, 0, 3.0, 0, 0],
```

```
    [3.5, 0, 0, 0, 3.5, 0, 0],
```

```
    [4.0, 0, 0, 0, 4.0, 0, 0],
```

```
    [4.5, 0, 0, 0, 4.5, 0, 0],
```

```
    [5.0, 0, 0, 0, 5.0, 0, 0]
```

```
]
```

```
# ----- Parameters for plot -----
```

```
scenario = f"at P = {(P / 1e6):.1f} MPa and T = {(T - 273.15):.1f} °C"
```

```
NaCl_conc = [ion_conc[0] for ion_conc in ion_conc_arr]
```

Thermodynamic Conditions

- Temperature (K)
- Pressure (Pa)

Array of ion concentrations (mol ion/kg brine)

- Ordered as Na, K, Mg, Ca, Cl, SO₄, and NO₃
- The example shown uses varying NaCl concentrations in the brine.

This description will appear in the plot and may be modified if needed.

Experimental Data

These data are displayed in the plots for comparison with the model results. Modification is optional.

```
# CO2 solubility at 0.1 MPa and 25 °C
markham_kobe = {
    "NaCl_mol_kg": [0, 0.1, 0.2, 0.5, 1.0, 2.0, 3.0, 4.0],
    "solubility_mol_mol": np.array([6.09, 5.95, 5.80, 5.39, 4.79, 3.88, 3.16, 2.65])*1e-4
}

yasunishi_yoshida = {
    "NaCl_mol_kg": [0.455, 1.000, 1.442, 1.945, 2.486, 3.098, 3.400, 4.216, 5.096],
    "solubility_mol_mol": np.array([5.42, 4.80, 4.31, 3.83, 3.44, 2.98, 2.75, 2.32, 1.91])*1e-4
}

# CO2-saturated water density at 0.1 MPa and 25 °C
moore = {
    "NaCl_mol_kg": 0,
    "density_kg_m3": 996.345
}
```

Reference:

Markham and Kobe (1941): [The Solubility of Carbon Dioxide and Nitrous Oxide in Aqueous Salt Solutions](#)

Yasunishi and Yoshida (1979): [Solubility of carbon dioxide in aqueous electrolyte solutions](#)

Moore et al. (1982): [Partial molar volumes of gases at infinite dilution in water at 298.15 K](#)

Sections That Do Not Require Modification

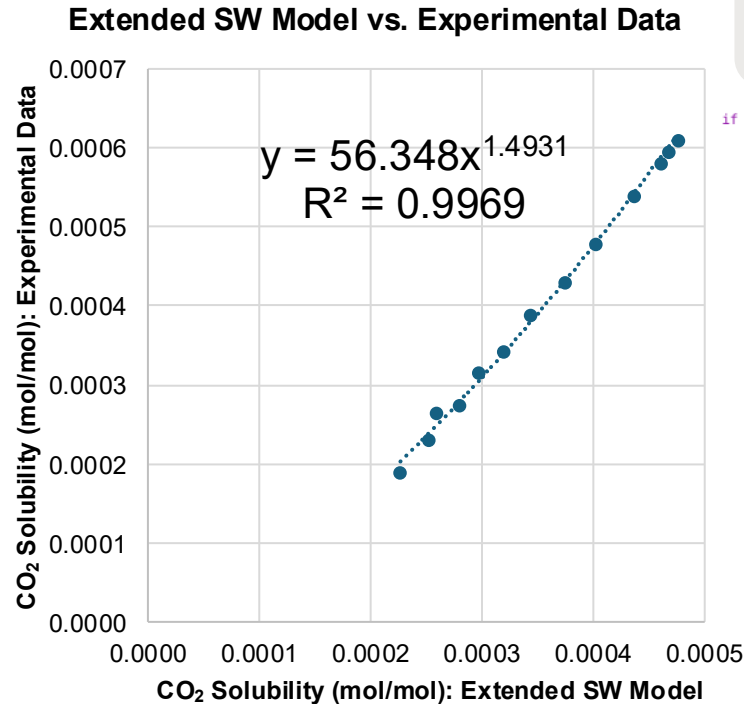
- **Model Parameters:** defines constants and correlations used throughout the calculations.
- **Helper Functions:** contains supporting functions used for thermodynamic and property calculations.
 - Flash and EOS
 - Solubility Calculation
 - Density Calculations
- **Main Code:** executes the workflow and connects all calculation steps.
- **Results and Plots:** generates numerical outputs and visualization plots for analysis.

These sections are preconfigured for standard use and typically do not require user modification.

Note on the Correction Factor for CO₂ Solubility

The correction factor is implemented in the following helper functions:

- *def calc_CO2_solubility*
- *def calc_solution_density*



```
if conv == 1:
    CO2_solubility = float(xi[0])
    mass_water = float(xi[1]) * Mw[1] / 1000 # kg

    # corrected solubility by experimental data
    CO2_solubility_corrected = float(xi[0])**1.4931 * 56.348
    mass_water_corrected = (1 - CO2_solubility_corrected) * Mw[1] / 1000 # kg

    m_CO2 = CO2_solubility / mass_water
    m_CO2_corrected = CO2_solubility_corrected / mass_water_corrected

    # calculate density of pure water and CO2-free brine
    rho_water = calc_pure_water_density(T, P) # g/cm3
    rho_brine = calc_brine_density(rho_water, m_s, Mw_s) # g/cm3

    # calculate CO2 partial molar volume
    dP = 1e-3 * P
    Zl1 = choose_physical_root(EOS, 'AQ', T_r_water, ion_conc, xi, T, Tc, Pc, omega, P, al, bl)
    fug_l1 = get_fug(EOS, 'AQ', T_r_water, ion_conc, xi, Zl1, Tc, Pc, P, T, omega, al, bl)
    Zl2 = choose_physical_root(EOS, 'AQ', T_r_water, ion_conc, xi, T, Tc, Pc, omega, P+dP, al, bl)
    fug_l2 = get_fug(EOS, 'AQ', T_r_water, ion_conc, xi, Zl2, Tc, Pc, P+dP, T, omega, al, bl)
    dln_fug = np.log(fug_l2[0]) - np.log(fug_l1[0])
    vbar_CO2 = R*T*(dln_fug/dP)

    # calculate density of CO2-sat brine
    rho_sol = (1 + m_s*Mw_s/1000 + m_CO2*Mw[0]/1000) / ((1 + m_s*Mw_s/1000)/(rho_brine*1000) + m_CO2*vbar_CO2)
    rho_sol_corrected = (1 + m_s*Mw_s/1000 + m_CO2_corrected*Mw[0]/1000) / ((1 + m_s*Mw_s/1000)/(rho_brine*1000) + m_CO2_corrected*vbar_CO2)
```